Lab 5: Ports, Interrupts, and (Bouncy) Switches

10/08/2026

Daniel Xu
ECE 3218L

1 Overview

Besides acting as plain inputs and outputs, the port pins of the ATmega328P can request interrupts. Port D pins 2 and 3 are the external interrupts INT0 and INT1, which can respond to a low level or any edge. Every port pin can also generate a pin change interrupt (PCINT) whenever its logic level changes.

This lab used both kinds. The first program toggles an LED from the INT0 interrupt each time a push button is pressed. The interrupt service routine (ISR) was given a software delay to debounce the button. The second program triggers a pin change interrupt from its own output pin once per second, and the ISR toggles a second LED so that the two LEDs alternate.

2 Procedure

Figure 1 shows the circuit. The push button connects Port D pin 2 (Digital 2) to ground. With the pin's internal pull-up enabled, the pin sits high by default, and a button press pulls it low. LED 1 is driven by Port B pin 0 (Digital 8) and LED 2 by Port C pin 0 (Analog 0). Both LEDs are active high: writing a 1 to the port bit turns the LED on.

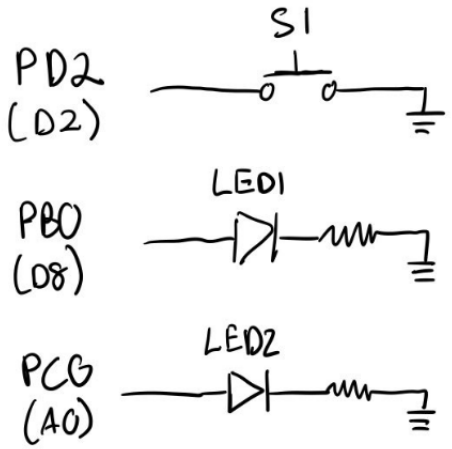


Figure 1: Lab circuit. This was built on the provided breakout board in lab, which had integrated switches, LEDs, and resistors.

1. **Program 1 without debouncing.** The program from the handout sets PB0 as an output, makes PD2 an input with its pull-up enabled, enables interrupts, and waits for an interrupt

while in the spin loop. Each time the ISR runs, it adds one to the 32-bit variable COUNT (SRAM 0x0102–0x0105), toggles PB0, and records the new LED state in LEDON (0x0100).

After the program had run for a few seconds, execution was paused and SRAM 0x0100–0x0105 was confirmed to be all zeros. The program was then resumed. The button was pressed firmly, held for about half a second, and released, and COUNT was read from memory. This was repeated for five presses. The data is shown in Table 1.

Table 1: COUNT after one press of about half a second, without debouncing.

Trial	COUNT (hex)	COUNT (decimal)
1	0x0005FD34	392500
2	0x0003774A	227146
3	0x00031379	201593
4	0x0003292C	207148
5	0x00038D25	232741
Average		252225.6

2. **Millisecond delay.** The lab manual provided wait1ms, which spends exactly 16000 clock cycles (1 ms at 16 MHz) including its call and return. A subroutine waitnms was written that calls wait1ms n times, with n passed in r25:r24 (Code 1). Since 1000 does not fit in 8 bits (the maximum is 255), the counter must be 2 bytes wide.

Code 1: The n-millisecond delay used for debouncing.

```
waitnms:
    push r24            ; 2
    push r25            ; 2
    push zl             ; 2
    push zh             ; 2
    cp r24,ZERO         ; 1
    cpc r25,ZERO        ; 1
    breq waitnmsdone    ; 1 if n != 0
waitnmsloop:
    rcall wait1ms       ; 16000 total with rcall and ret
    sbiw r25:r24,1      ; 2
    brne waitnmsloop    ; 2 if taken, 1 if not
waitnmsdone:
    pop zh              ; 2
    pop zl              ; 2
    pop r25             ; 2
    pop r24             ; 2
    ret                 ; 4
```

From the rcall to the ret, the subroutine takes $16004n + 25$ cycles, which predicts 16029, 160065, and 16004025 cycles for $n = 1$, 10, and 1000, so a 1000 ms request lasts 1.00025 s.

3. **Program 1 with debouncing.** A delay was added as the first step of the INT0 ISR so that the switch stops bouncing before the LED is toggled:

Code 2: Start of the debounced INT0 ISR.

```
EXT_INT0:
    ldi r24, low(150)    ; wait 150 ms
    ldi r25, high(150)
    rcall waitnms
```

A 500 ms delay was tried first. It debounced the button, but the LED responded noticeably late. The delay was then shortened to 150 ms. With this delay, brief taps toggled the LED once, but holding the button made the LED switch on and off every 150 ms. Since a normal press is a brief tap, 150 ms was kept. To check it, the button was pressed 10 times, and COUNT read 0x0000000A, equal to the number of presses.

4. **Program 2: pin change interrupt.** The second program makes PC0 an output and sets it high, so LED 2 starts on and LED 1 starts off. It enables pin change interrupt group 1 (Port C) in PCICR, unmasks only PCINT8 (PC0) in PCMSK1, and clears any pending flag in PCIFR.

Code 3: Pin change interrupt setup for PC0.

```
    ldi r16,(1<<PCIE1)  ; enable pin change group 1 (Port C)
    sts PCICR,r16
    ldi r16,(1<<PCINT8) ; only PC0 in that group
    sts PCMSK1,r16
    ldi r16,(1<<PCIF1)  ; clear a stale request (writing 1 clears
       it)
    out PCIFR,r16
```

The main loop waits 1000 ms and then toggles PC0, using the variable PCLH to remember the pin's state. Each toggle is a pin change on PCINT8, so it requests the PCINT1 interrupt. The PCINT1 ISR increments COUNT and toggles PB0 using LEDON, the same way as the ISR in Program 1. On the board, the two LEDs alternated, each on for one second while the other was off.

3 Discussion

3.1 Why COUNT Was So Large

A single press of the button increased COUNT by hundreds of thousands rather than by one (Table 1). The LED turned on as soon as the button was pressed, looked dimmer while the button was held, and became brighter when it was released. After release, it stayed on in some trials and off in others. COUNT is the number of times the INT0 ISR ran.

Part of the count comes from switch bounce: when the contacts close, they bounce open and closed for a few milliseconds before settling, and each closure pulls PD2 low. However, the main reason the count is so large is the trigger mode. Writing 0 to EICRA selects a *low-level* interrupt, which is requested continuously for as long as PD2 is low, not once per falling edge. This means that as long as the button is held, the interrupt will keep running and COUNT will increment.

The LED being dimmer while held makes sense, since the ISR kept running, which would quickly toggle the LED on and off, making it seem dimmer. When released, it would unpredictably be left on or off depending on whether the ISR ran an odd or even number of times.

Table 2 counts the cycles for one pass through the original ISR using the instruction timings from the AVR Instruction Set Manual.

Table 2: Clock cycles for one pass through the INT0 ISR without the delay.

	Cycles	
Code	LED off → on	LED on → off
(interrupt response)	4	4
jmp EXT_INT0	3	3
lds r27,COUNT	2	2
lds r26,COUNT+1	2	2
lds r25,COUNT+2	2	2
lds r24,COUNT+3	2	2
adiw r25:r24,1	2	2
brcc NOUHINC	2	2
lds r17,LEDON	2	2
tst r17	1	1
brne ITSON	1	2
sbi PORTB,0	2	–
sbr r17,1	1	–
rjmp DUNIT	2	–
cbi PORTB,0	–	2
clr r17	–	1
sts LEDON,r17	2	2
sts COUNT,r27	2	2
sts COUNT+1,r26	2	2
sts COUNT+2,r25	2	2
sts COUNT+3,r24	2	2
reti	4	4
Total in the ISR	42	41
rjmp SPIN	2	2
Total per count	44	43

When the LED goes from off to on, the toggle branch takes 6 cycles and one pass through the ISR takes 42 cycles, or 2.6 μs at 16 MHz. Multiplying the average count by the time per pass estimates how long the ISR ran during one press:

$$t \approx \text{COUNT} \times \frac{42}{16 \times 10^6 \text{ Hz}} = 252225.6 \times \frac{42}{16 \times 10^6 \text{ Hz}} \approx 0.662 \text{ s}. \tag{1}$$

Counting the 2-cycle rjmp SPIN that runs between passes raises the cost to 44 cycles per count and the estimate to 0.694 s. The button press was timed by hand, which explains the higher-than-expected number.

This estimate only measures how long PD2 was low in total. Measuring the average time between bounces needs two changes to the program. First, INT0 should trigger on a falling edge (ISC01 = 1 in EICRA) instead of a low level, so each bounce produces exactly one interrupt and COUNT becomes the number of bounces. Second, another counter should be incremented inside the SPIN loop, so the program also knows how many times the main loop ran between interrupts. Each pass of the SPIN loop and of the ISR takes a known number of cycles, so the total elapsed time can be found through a simple calculation similar to the one above.

3.2 Debouncing in Software

Bouncing is usually removed in hardware with an SR latch or a Schmitt trigger. Here it was handled in software by waiting at the start of the ISR. The interrupt is not re-entered during the delay, because the processor clears the global interrupt flag when it enters an ISR and only reti sets it again. By the time the delay ends, the contacts have settled, and the LED is toggled once. If the button is held and PD2 is still low, then the ISR will trigger a second time, which explains the toggling LED when held with the 150 ms delay.

3.3 Why Program 2 Needs No Delay

The PCINT1 ISR needs no debounce delay because nothing mechanical drives PC0. The pin is an output written by the processor itself, so each toggle is one clean transition and produces exactly one pin change interrupt. The ISR also needs no timer of its own, because the program is entirely software-controlled: the 1 s delay already runs in the main loop, and the ISR only reacts to the edge the main loop creates. A pin change interrupt fires on both rising and falling edges, which is why the ISR runs on every toggle of PC0, in both directions.

3.4 Why Use a Self-Triggered Interrupt?

The AVR has no software interrupt instruction, so a self-triggered pin change interrupt is a way to simulate one. A subroutine call runs the code as part of the main program, while the interrupt runs it as an ISR, with other interrupts held off until it returns.

Using a pin change interrupt also makes sense if the same routine needs to be triggered by both software and hardware. An external device could trigger it by changing the pin's level, while software could trigger it the same way this lab did.

Toggling a pin to trigger an interrupt also makes sense when the ISR needs to be tested on the Uno before the external hardware is connected.

4 Conclusion

This lab configured ATmega328P port pins to generate interrupts from both an external event and the program's own output. One half-second button press ran the ISR an average of 252225.6 times, because the interrupt repeats for as long as the pin is low. At 42 clock cycles per pass, that count corresponds to about 0.66 s of the pin being held low, which makes sense given human error in counting time. Adding a 150 ms delay at the start of the ISR made each brief tap toggle the LED once, and after 10 presses COUNT read 0x0000000A. The second program toggled PC0 once per second to trigger an edge-triggered pin change interrupt, and its ISR alternated the second LED without any debouncing, because a pin driven by the processor does not bounce.

A Full Code

Code 4: Program 1. Toggle the LED with a debounced push button.

```
;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
;
; Lab 5: Interrupts and (bouncy) switches
;
;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
```

```
.DEF ZERO=R0

.dseg                   ; SRAM variables
LEDON:  .byte 1         ; LED status ~0 ==> ON, 0 ==> OFF
PCLH:   .byte 1         ; Port C pin 0 status 1 ==> High, 0 ==> Low for PCINT0
COUNT:  .byte 4         ; number of times the interrupt routine was executed

; interrupt table
; this program requires RESET, INT0, and PCINT1
.cseg
    jmp RESET           ; Reset Handler
    jmp EXT_INT0        ; IRQ0 Handler (external hardware interrupt)
;.org PCI1addr          ; This is used in part 4, the timed lights, comment it out here
;   jmp PC_INT1         ; PCINT1 Handler (pin change interrupt on Port B) for part 4

RESET:
main:
    ; initialize stack pointer
    ldi r16,high(RAMEND)
    out sph,r16
    ldi r16,low(RAMEND)
    out spl,r16
    ; (R0)<-0x00 aliased to ZERO
    clr ZERO
    ; go to program
    ;rcall TESTNMS      ; test waitnms for prelab
    rcall L06P01

;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
;
; Lab 5 program 1: Toggle LED with momentary push-button switch
;
;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
L06P01:
    ; initialize variables
    ; LED status flag
    sts LEDON,ZERO      ; 0 ==> LED is not on
    sts LEDON+1,ZERO    ; clear this byte to make the next 4 easier to see
    ; number of times the ISR has been executed
    sts COUNT,ZERO      ; clear ISR count high byte
    sts COUNT+1,ZERO    ; clear ISR count upper middle byte
    sts COUNT+2,ZERO    ; clear ISR count lower middle byte
    sts COUNT+3,ZERO    ; clear ISR count low byte

    ; set up port B for output to LED
    sbi DDRB,0          ; bit 0 in Port B DDR <- 1, data direction out
    cbi PORTB,0         ; bit 0 in Port B <- 0, turn off the LED

    ; set up hardware external interrupt INT0 to trigger when the Arduino board
    ; digital pin 2 (aliased to both INT0 and Port D pin 2) is pulled low
    ; set up Port D pin 2 for input with pull-up to logic level high
    cbi DDRD,2          ; (a) set it to input
    sbi PORTD,2         ; (b) engage the 5v pull-up

    ; Now set up for a hardware interrupt on INT0 (momentary switch)
    ; INT0 is on the same pin (Port D pin 2)
```

```
62      sts EICRA,r0          ; clear bits 0,1 so that IRQ is generated when pin is
            pulled low
63      sbi EIMSK,0           ; enable hardware interrupt INT0; bit 0 <- 1
64      sei                   ; enable interrupts globally
65
66      ; wait here for interrupt (Hit break, ||, in debug mode to pause execution
            here)
67  SPIN:
68      rjmp SPIN
69
70  ; ISR for external interrupt 0 (INT0)
71  EXT_INT0:
72      ldi r24, low(150)     ; wait 150 ms
73      ldi r25, high(150)
74      rcall waitnms
75      ; load count of number of times ISR has been executed and increment it
76      lds r27,COUNT         ; load the current ISR count, high byte
77      lds r26,COUNT+1       ; load the current ISR count, upper middle byte
78      lds r25,COUNT+2       ; load the current ISR count, lower middle byte
79      lds r24,COUNT+3       ; load the current ISR count, low byte
80      adiw r25:r24,1        ; increment the count
81      brcc NOUHINC          ; if C==0, nothing to add to the upper half
82      adiw r27:r26,1        ; else add one to upper half
83  NOUHINC:
84      ; toggle the LED state
85      lds r17,LEDON         ; get the LED status in r17
86      tst r17               ; is it zero?
87      brne ITSON            ; no? then the LED is on; go turn it off
88      sbi PORTB,0           ; yes? then it's off; turn it on
89      sbr r17,1             ; set the flag in r17
90      rjmp DUNIT
91  ITSON:
92      cbi PORTB,0           ; turn it off
93      clr r17               ; clear the flag in r17
94  DUNIT:
95      sts LEDON,r17         ; save r17 to the LED status flag
96      ; save the ISR execution count
97      sts COUNT,r27         ; update the ISR count, high byte
98      sts COUNT+1,r26       ; update the ISR count, upper middle byte
99      sts COUNT+2,r25       ; update the ISR count, lower middle byte
100     sts COUNT+3,r24       ; update the ISR count, low byte
101     reti
102
103 ; 1 msec timer subroutine (exact; time includes call and return)
104 ; wait1ms expends 16000 clock cycles.
105 ; rcall wait1ms uses 3 clock cycles
106 ; Therefore from the rcall in the calling program through the
107 ; ret in the subroutine wait1ms expends
108 ; 3+1+1+1+(3996*4)[in loop]+3[end of loop]+1+1+1+4 = 16000cc
109 ; which at 16MHz (cc/s) equals 1ms.
110 wait1ms:
111     ldi zh,0x0F           ; 1cc
112     ldi zl,0x9D           ; 1cc
113     clz                   ; 1cc
114 waitloop:                 ; countdown 0x0F9D = 3997dec
115     sbiw z,1              ; 2cc
116     brne waitloop         ; 2cc true, 1cc false
117     nop                   ; 1cc
118     nop                   ; 1cc
```

```
119     nop                 ; 1cc
120     ret                 ; 4cc
121
122 ; waitnms: wait n ms
123 ; load n into r24, r25
124 ; r24, r25 and Z are restored before returning
125 waitnms:
126     push r24            ; 2
127     push r25            ; 2
128     push zl             ; 2
129     push zh             ; 2
130     cp r24,ZERO         ; 1
131     cpc r25,ZERO        ; 1
132     breq waitnmsdone    ; 1 if n != 0
133 waitnmsloop:
134     rcall wait1ms       ; 16000 total with rcall and ret
135     sbiw r25:r24,1      ; 2
136     brne waitnmsloop    ; 2 if taken, 1 if not
137 waitnmsdone:
138     pop zh              ; 2
139     pop zl              ; 2
140     pop r25             ; 2
141     pop r24             ; 2
142     ret                 ; 4
143
144 ; test waitnms
145 ; n = 1: 16029, n = 10: 160065, n = 1000: 16004025 (about 1.00025 s)
146 TESTNMS:
147     ldi r24,low(1)
148     ldi r25,high(1)
149     rcall waitnms
150     ldi r24,low(10)
151     ldi r25,high(10)
152     rcall waitnms
153     ldi r24,low(1000)
154     ldi r25,high(1000)
155     rcall waitnms
156     ret
```

Code 5: Program 2. Alternating LEDs with a pin change interrupt.

```
1  ; Lab 5, Program 2: alternating LEDs with a pin change interrupt
2  ; Once a second the main loop flips PC0 (Uno A0, PCINT8). That change
3  ; requests the PCINT1 interrupt, and its ISR flips the LED on PB0
4  ; (Uno pin 8), so one LED is always on while the other is off.
5
6  .DEF ZERO=R0
7
8  .dseg                       ; SRAM variables
9  LEDON:  .byte 1             ; LED status ~0 ==> ON, 0 ==> OFF
10 PCLH:   .byte 1             ; Port C pin 0 status 1 ==> High, 0 ==> Low for
       PCINT0
11 COUNT:  .byte 4             ; number of times the interrupt routine was
       executed
12
13 ; interrupt table
14 ; this program requires RESET and PCINT1
15 .cseg
```

```
.org 0x0000
        jmp RESET           ; Reset Handler
.org PCI1addr
        jmp PC_INT1         ; PCINT1 Handler

.org INT_VECTORS_SIZE       ; code starts after the vector table
RESET:
        ; initialize stack pointer
        ldi r16,high(RAMEND)
        out SPH,r16
        ldi r16,low(RAMEND)
        out SPL,r16
        ; (R0)<-0x00 aliased to ZERO
        clr ZERO

        ; initialize variables
        ; LED status flag
        sts LEDON,ZERO      ; 0 ==> LED is not on
        ; number of times the ISR has been executed
        sts COUNT,ZERO      ; clear ISR count high byte
        sts COUNT+1,ZERO    ; clear ISR count upper middle byte
        sts COUNT+2,ZERO    ; clear ISR count lower middle byte
        sts COUNT+3,ZERO    ; clear ISR count low byte

        ; set up port B for output to LED
        sbi DDRB,0          ; bit 0 in Port B DDR <- 1, data direction out
        cbi PORTB,0         ; bit 0 in Port B <- 0, turn off the LED

        ; LED 2 on PC0 starts on
        sbi DDRC,0
        sbi PORTC,0
        ldi r16,1
        sts PCLH,r16        ; remember that PC0 is high

        ; let a change on PC0 (PCINT8) request PCINT1
        ; sbi/out can't reach PCICR and PCMSK1, so use sts
        ldi r16,(1<<PCIE1)  ; enable pin change group 1 (Port C)
        sts PCICR,r16
        ldi r16,(1<<PCINT8) ; only PC0 in that group
        sts PCMSK1,r16
        ldi r16,(1<<PCIF1)  ; clear a stale request (writing 1 clears it)
        out PCIFR,r16

        sei                 ; enable interrupts globally

; main loop: wait 1 s, then flip PC0, which triggers the interrupt
MAINLOOP:
        ldi r28,low(1000)   ; Y = 1000 ms
        ldi r29,high(1000)  ; 1000 needs 16 bits
        rcall waitNms

        ; flip PC0
        lds r16,PCLH
        tst r16
        brne PCHIGH         ; PC0 high -> low
        sbi PORTC,0         ; PC0 low -> high (LED 2 on)
        ldi r16,1
        rjmp PCSAVE
PCHIGH:
```

```
75          cbi PORTC,0          ; LED 2 off
76          clr r16
77  PCSAVE:
78          sts PCLH,r16
79          rjmp MAINLOOP
80
81  ; PCINT1 ISR: count the interrupt, then toggle the LED on PB0
82  ; saves SREG and every register it uses, since it interrupts the main loop
83  PC_INT1:
84          push r16
85          in r16,SREG
86          push r16
87          push r17
88          push r24
89          push r25
90          push r26
91          push r27
92
93          ; load count of number of times ISR has been executed and increment it
94          lds r27,COUNT        ; load the current ISR count, high byte
95          lds r26,COUNT+1      ; load the current ISR count, upper middle byte
96          lds r25,COUNT+2      ; load the current ISR count, lower middle byte
97          lds r24,COUNT+3      ; load the current ISR count, low byte
98          adiw r25:r24,1       ; increment the count
99          brcc NOUHINC         ; if C==0, nothing to add to the upper half
100         adiw r27:r26,1       ; else add one to upper half
101 NOUHINC:
102         ; save the ISR execution count
103         sts COUNT,r27        ; update the ISR count, high byte
104         sts COUNT+1,r26      ; update the ISR count, upper middle byte
105         sts COUNT+2,r25      ; update the ISR count, lower middle byte
106         sts COUNT+3,r24      ; update the ISR count, low byte
107
108         ; toggle the LED state
109         lds r17,LEDON        ; get the LED status in r17
110         tst r17              ; is it zero?
111         brne ITSON           ; no? then the LED is on; go turn it off
112         sbi PORTB,0          ; yes? then it's off; turn it on
113         ldi r17,1            ; set the flag in r17
114         rjmp DUNIT
115 ITSON:
116         cbi PORTB,0          ; turn it off
117         clr r17              ; clear the flag in r17
118 DUNIT:
119         sts LEDON,r17        ; save r17 to the LED status flag
120
121         ; restore in reverse order
122         pop r27
123         pop r26
124         pop r25
125         pop r24
126         pop r17
127         pop r16
128         out SREG,r16
129         pop r16
130         reti
131
132 ; waitNms: wait n ms
133 ; load n (at least 1) into Y (r29:r28)
```

```
; Y ends at 0; Z is restored before returning
waitNms:
        push r30            ; wait1ms changes Z
        push r31
WNLOOP:
        rcall wait1ms       ; 16000 total with rcall and ret
        sbiw r29:r28,1
        brne WNLOOP
        pop r31
        pop r30
        ret

; 1 msec timer subroutine (exact; time includes call and return)
; wait1ms expends 16000 clock cycles.
; rcall wait1ms uses 3 clock cycles
; Therefore from the rcall in the calling program through the
; ret in the subroutine wait1ms expends
; 3+1+1+1+(3996*4)[in loop]+3[end of loop]+1+1+1+4 = 16000cc
; which at 16MHz (cc/s) equals 1ms.
wait1ms:
        ldi zh,0x0F         ; 1cc
        ldi zl,0x9D         ; 1cc
        clz                 ; 1cc
waitloop:                   ; countdown 0x0F9D = 3997dec
        sbiw r31:r30,1      ; 2cc
        brne waitloop       ; 2cc true, 1cc false
        nop                 ; 1cc
        nop                 ; 1cc
        nop                 ; 1cc
        ret                 ; 4cc
```